NeuroMF: Latent MeanFlow for One-Step 3D Brain MRI Synthesis

Technical Report — Detailed Scientific Summary

Mario Pascual-González

February 2026

Phases 0–5 complete (code); Phase 4 best model trained and evaluated; Phase 5 generation pipeline ready

---

Contents

---

1 Motivation and Problem Statement

1.1 Clinical Need

Generative models for 3D brain MRI synthesis serve three clinical purposes: (i) data augmentation for rare pathologies with scarce training data, (ii) synthetic cohorts for privacy-preserving research, and (iii) counterfactual generation for explainability. The critical bottleneck in existing methods is **sampling cost**: state-of-the-art approaches (DDPM, flow matching, rectified flow) require 5–1000 network evaluations per volume, making large-scale synthesis prohibitively slow.

1.2 Gap in the Literature

No prior work has applied MeanFlow—or any 1-step flow-based model—to 3D medical image synthesis. The closest works are summarised in Table 1.

Table 1: Comparison with related generative methods for brain MRI.

Method	Space	NFE	Paradigm	Domain
MAISI-v2 [5]	Latent	5–50	Rectified Flow	3D CT/MRI
MOTFM [7]	Pixel	10–50	OT Flow Matching	3D Brain MRI
Med-DDPM [8]	Pixel	1000	DDPM	3D Brain MRI
pMF [3]	Pixel	1	Progressive MeanFlow	2D Natural Images
NeuroMF (ours)	**Latent**	**1**	**MeanFlow (iMF dual-head)**	**3D Brain MRI**

Our work fills the intersection: **1-step + latent + 3D + medical**.

1.3 Approach Overview

NeuroMF trains a MeanFlow model in the latent space of a frozen MAISI 3D VAE:

```
Input MRI (1x192^3) --> Frozen MAISI VAE Encoder --> Latent (4x48^3)
                                                        |
                                                  Train MeanFlow
                                                        |
                                                        v
Synthetic MRI (1x192^3) <-- Frozen MAISI VAE Decoder <--'
```

At inference, a **single forward pass** through the MeanFlow network generates a complete 3D brain MRI volume. The network learns the *average velocity* of a probability flow ODE, which by construction encodes the entire transport from noise to data in one evaluation.

2 Theoretical Foundations

2.1 Notation and Tensor Dimensions

Table 2 defines all symbols used in this report and their tensor shapes as instantiated in code (`src/neuromf/`). Equations are presented *per-sample* (batch index suppressed) unless stated otherwise; grey *Shapes* boxes show the batched implementation dimensions.

Broadcasting convention. When a scalar-per-sample quantity $t \in \mathbb{R}^B$ multiplies a 5-D tensor $z \in \mathbb{R}^{B\times C\times D\times H\times W}$, t is reshaped to $(B,1,1,1,1)$ via `t.view(-1, 1, 1, 1, 1)` before

Table 2: Notation, symbols, and tensor dimensions. All latent-space tensors live in $\mathbb{R}^{C\times D\times H\times W}$ with C=4, D=H=W=48. The total latent dimensionality is $d = C\cdot D\cdot H\cdot W = 4\times 48^3 = 442{,}368$.

Symbol	Meaning	Per-sample shape	Batched shape
z_0	Clean latent (data)	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
ε	Gaussian noise	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
z_t	Noisy interpolation at time t	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
v_c	Conditional velocity (target)	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
u_θ	Average velocity (model output)	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
$\hat{x}$	Denoised data estimate (x-pred)	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
$\hat{v}$	v-head output (tangent predictor)	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
$\tilde{v}$	JVP tangent vector	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
V_θ	Compound velocity	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
$\frac{du}{dt}$	JVP output (directional deriv.)	$\mathbb{R}^{C\times D\times H\times W}$	$(B, 4, 48, 48, 48)$
t	Upper time (current)	$[0, 1]$	$(B,)$
r	Lower time (interval start)	$[0, 1],\ r \le t$	$(B,)$
h	Interval width $h \coloneqq t - r$	$[0, 1]$	$(B,)$
$\mathbf{e}_t$	Time embedding for t	$\mathbb{R}^{d_{\text{emb}}}$	$(B, 256)$
$\mathbf{e}_h$	Time embedding for h	$\mathbb{R}^{d_{\text{emb}}}$	$(B, 256)$
$\mathbf{e}$	Combined embedding $\mathbf{e}_t + \mathbf{e}_h$	$\mathbb{R}^{d_{\text{emb}}}$	$(B, 256)$
ℓ_u, ℓ_v	Per-sample raw losses	$\mathbb{R}_{\ge 0}$	$(B,)$
w_u, w_v	Per-sample adaptive weights	$\mathbb{R}_{>0}$	$(B,)$
$\mathcal{L}$	Total scalar loss	$\mathbb{R}$	scalar

element-wise multiplication. We write this broadcast as $t \odot z$ and denote the broadcast shape as $\bar{t} \in \mathbb{R}^{B\times 1\times 1\times 1\times 1}$.

2.2 Operator Definitions

We define two operators used throughout this report.

Definition 1 (Stop-gradient). The stop-gradient operator $\text{sg}[\cdot]$ returns the value of its argument but blocks gradient propagation during backpropagation:

$$\text{sg}[x] \coloneqq x, \qquad \frac{\partial}{\partial \theta}\,\text{sg}[x(\theta)] = \mathbf{0} \tag{1}$$

In code, $\text{sg}[\cdot]$ is implemented as `.detach()`, which detaches the tensor from the computation graph. For any differentiable function $f(\theta, \text{sg}[g(\theta)])$, the gradient $\nabla_\theta f$ treats $\text{sg}[g]$ as a constant, so only the explicit dependence of f on θ is differentiated.

Definition 2 (Jacobian-Vector Product). Given a differentiable function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ evaluated at a point $\mathbf{x}_0 \in \mathbb{R}^n$, and a tangent vector $\mathbf{v} \in \mathbb{R}^n$, the Jacobian-vector product (JVP) is:

$$\text{JVP}(\mathbf{f};\ \mathbf{x}_0;\ \mathbf{v}) \coloneqq J_{\mathbf{f}}(\mathbf{x}_0)\,\mathbf{v} = \frac{d}{d\alpha}\,\mathbf{f}(\mathbf{x}_0 + \alpha\,\mathbf{v})\Big|_{\alpha=0} \in \mathbb{R}^m \tag{2}$$

where $J_{\mathbf{f}}(\mathbf{x}_0) \in \mathbb{R}^{m\times n}$ is the Jacobian matrix. The JVP can be computed in $O(n)$ time via forward-mode automatic differentiation (`torch.func.jvp`), without ever constructing the full $O(n^2)$ Jacobian.

In our setting, $\mathbf{f} = u_\theta$ maps $(\underbrace{z_t}_{\mathbb{R}^d},\ \underbrace{t}_{\mathbb{R}},\ \underbrace{r}_{\mathbb{R}})$ to $\mathbb{R}^d$ with $d = 442{,}368$, so the Jacobian $J \in \mathbb{R}^{442{,}368\times 442{,}370}$ is never formed. The tangent vector is $(\tilde{v},\ 1,\ 0)$ (see §2.5).

2.3 Flow Matching Preliminaries

Flow matching [9, 10] defines a probability path between data and noise via linear interpolation. For a single sample:

$$z_t = (1-t)\, z_0 + t\, \varepsilon, \qquad z_0 \sim p_{\text{data}}, \quad \varepsilon \sim \mathcal{N}(\mathbf{0},\, I_d), \quad t \in [0,1] \tag{3}$$

$z_0, \varepsilon, z_t \in \mathbb{R}^{C\times D\times H\times W}$; batched: $(B, 4, 48, 48, 48)$. I_d is the $d \times d$ identity with $d = 442{,}368$. $t \in \mathbb{R}$; batched: $(B,)$, broadcast to $(B, 1, 1, 1, 1)$.
Code: `meanflow_loss.py:201`: `z_t = (1 - t_broad) * z_0 + t_broad * eps` where `t_broad = t.view(-1, 1, 1, 1, 1)`.

where t=0 is data and t=1 is noise. The **conditional velocity field** is defined as:

$$v_c(z_t, t) \coloneqq \varepsilon - z_0 \quad \in \mathbb{R}^{C\times D\times H\times W} \tag{4}$$

$v_c \in \mathbb{R}^{C\times D\times H\times W}$; batched: $(B, 4, 48, 48, 48)$.
Code: `meanflow_loss.py:204`: `v_c = eps - z_0`.

A neural network v_θ is trained to match this field:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{z_0,\, \varepsilon,\, t}\left[\, \left\| v_\theta(z_t, t) - v_c \right\|_2^2 \right] \tag{5}$$

where $\|\cdot\|_2^2$ denotes the squared L_2 norm summed over all $d = C \cdot D \cdot H \cdot W = 442{,}368$ elements.

Limitation. Sampling requires integrating the learned velocity field from t=1 to t=0 via numerical ODE solvers, requiring $K \geq 5$ network evaluations even with trajectory straightening (rectified flow).

2.4 MeanFlow: Average Velocity for 1-Step Generation

MeanFlow [1] replaces the instantaneous velocity $v(z_t, t)$ with the **average velocity** over an interval $[r, t]$:

$$u(z_t, r, t) \coloneqq \frac{1}{t-r} \int_r^t v(z_s, s)\, ds \quad \in \mathbb{R}^{C\times D\times H\times W} \tag{6}$$

$u \in \mathbb{R}^{C\times D\times H\times W}$; batched: $(B, 4, 48, 48, 48)$. $r, t \in \mathbb{R}$; batched: $(B,)$.

If the average velocity from t=0 to t=1 is known exactly, the entire flow can be computed in one step:

$$z_0 = z_1 - u_\theta(z_1,\, r{=}0,\, t{=}1) \quad \in \mathbb{R}^{C\times D\times H\times W} \qquad \text{[1-NFE generation]} \tag{7}$$

$z_1 = \varepsilon \sim \mathcal{N}(\mathbf{0}, I_d)$: $(B, 4, 48, 48, 48)$. $r = \mathbf{0}_B$, $t = \mathbf{1}_B$: each $(B,)$.
Code: `one_step.py:37`: `output = model(noise, r, t)`.

2.5 The MeanFlow Identity and Compound Velocity

The average velocity u must satisfy a self-consistency condition. Differentiating the integral definition (6) with respect to t and applying the chain rule yields the **MeanFlow Identity**:

$$v(z_t, t) = u(z_t, r, t) + (t-r)\left[\underbrace{\frac{\partial u}{\partial z_t}\, v(z_t, t)}_{\in \mathbb{R}^d,\ \text{spatial JVP}} + \underbrace{\frac{\partial u}{\partial t}}_{\in \mathbb{R}^d,\ \text{temporal deriv.}}\right] \tag{8}$$

All terms $\in \mathbb{R}^{C\times D\times H\times W}$; batched: $(B, 4, 48, 48, 48)$.
$\frac{\partial u}{\partial z_t} \in \mathbb{R}^{d\times d}$ is the Jacobian (never formed); only its product with v is computed via JVP.
$(t - r) \in \mathbb{R}$; batched: $(B,)$, broadcast to $(B, 1, 1, 1, 1)$.

The right-hand side, evaluated with the neural approximation u_θ, gives the **compound velocity**. We first compute the directional derivative (JVP) of u_θ with respect to its inputs (z_t, t, r) in the tangent direction $(\tilde{v}, 1, 0)$:

$$\left.\frac{du_\theta}{dt}\right|_{\tilde{v}} \coloneqq \mathrm{JVP}(u_\theta;\ (z_t, t, r);\ (\tilde{v}, 1, 0)) = \frac{\partial u_\theta}{\partial z_t}\,\tilde{v} + \frac{\partial u_\theta}{\partial t}\cdot 1 + \frac{\partial u_\theta}{\partial r}\cdot 0 \ \in \mathbb{R}^{C\times D\times H\times W} \tag{9}$$

Primals: $z_t \in \mathbb{R}^{B\times 4\times 48\times 48\times 48}$, $t \in \mathbb{R}^B$, $r \in \mathbb{R}^B$.
Tangents: $\tilde{v} \in \mathbb{R}^{B\times 4\times 48\times 48\times 48}$, $\mathbf{1}_B \in \mathbb{R}^B$, $\mathbf{0}_B \in \mathbb{R}^B$.
Output: $\frac{du}{dt} \in \mathbb{R}^{B\times 4\times 48\times 48\times 48}$.
Code: `jvp_strategies.py:117`: `u, du_dt, v = torch.func.jvp( _u_with_v_aux, (z_t, t, r), (v_tangent, dt, dr), has_aux=True)`.
Here `dt = torch.ones_like(t)`: $(B,)$; `dr = torch.zeros_like(r)`: $(B,)$.

The compound velocity is then:

$$\boxed{V_\theta \coloneqq u_\theta + (t - r)\cdot \mathrm{sg}\left[\left.\frac{du_\theta}{dt}\right|_{\tilde{v}}\right]} \quad \in \mathbb{R}^{C\times D\times H\times W} \tag{10}$$

$u_\theta, V_\theta \in \mathbb{R}^{B\times 4\times 48\times 48\times 48}$; $\mathrm{sg}[\cdot]$ is `.detach()` (Def. 1, Eq. 1).
$(t - r)$: $(B,)$ broadcast to $(B, 1, 1, 1, 1)$ via `.view(-1,1,1,1,1)`.
Code: `jvp_strategies.py:59`: `V = u + t_minus_r * du_dt.detach()`.
Gradients flow through u_θ but **not** through du/dt (stop-gradient).

The sg[·] on the JVP term is essential: it ensures that during backpropagation, gradients flow only through u_θ (left term), not through the JVP correction. Without sg[·], second-order derivatives of u_θ would be needed, making training intractable.

Tangent vector $\tilde{v}$. In the improved MeanFlow (iMF) formulation [2], the tangent $\tilde{v}$ is the model's own instantaneous velocity estimate at $r{=}t$ (see §2.6 for the dual-head variant):

$$\tilde{v} = u_\theta(z_t,\, r{=}t,\, t) \quad \in \mathbb{R}^{C\times D\times H\times W} \tag{11}$$

This makes V_θ depend only on z_t (not on ground-truth z_0 or ε), matching the inference setting where data is unavailable.

Special cases. When $r = t$ (flow-matching samples, 50% of the batch), $(t - r) = 0$ and $V_\theta = u_\theta$, recovering standard flow matching. When $r < t$ (MeanFlow samples), the JVP correction enforces self-consistency of the average velocity across intervals.

2.6 Improved MeanFlow (iMF) and the Dual-Head Architecture

The original MeanFlow uses the ground-truth velocity $v_c = \varepsilon - z_0$ as the JVP tangent. **Problem:** this creates a dependency on data at inference time (where z_0 is unavailable). The improved MeanFlow (iMF) [2] resolves this by using the model's own prediction as the tangent, making V a function of z_t alone.

The dual-head extension (our architecture, inspired by iMF) introduces two output heads on a shared UNet backbone:

- **u-head** (main): Predicts the denoised data estimate $\hat{x}$ (x-prediction mode), from which u_θ is derived. Used at inference.

- **v-head** (auxiliary): Predicts the instantaneous velocity $\hat{v}$, directly supervised against v_c. Provides the JVP tangent $\tilde{v} = \hat{v}$ during training. **Disabled at inference** (zero cost).

The shared backbone produces a feature tensor $\mathbf{h} \in \mathbb{R}^{B\times 64\times 48\times 48\times 48}$ (channel dimension equals `channels[0]`$= 64$). Both heads branch from $\mathbf{h}$:

$$\hat{x} = \text{UNet.out}(\mathbf{h}) \ \in \mathbb{R}^{C\times D\times H\times W}, \qquad \hat{v} = \text{v\_out}(\mathbf{h}) \ \in \mathbb{R}^{C\times D\times H\times W} \tag{12}$$

$\mathbf{h}$: $(B, 64, 48, 48, 48)$ — shared backbone final feature map.
$\hat{x}$: $(B, 4, 48, 48, 48)$ via `self.unet.out(h)` (Conv3d $64 \to 4$).
$\hat{v}$: $(B, 4, 48, 48, 48)$ via `self.v_out(h)` (ResBlock $\to$ GN $\to$ SiLU $\to$ Conv3d $64 \to 4$).
Code: `maisi_unet.py:415-418`.

The v-head solves a critical practical problem: without it, the tangent comes from the u-head's own prediction, which is poor in early training, creating a bootstrapping problem that causes loss divergence. The v-head receives direct supervision ($\|\hat{v} - v_c\|^p$), providing high-quality tangents from the first epoch. In our training, the v-head cosine alignment $\cos(\hat{v}, v_c)$ reached 0.39 by epoch 50—85% of its final value of 0.44.

2.7 x-Prediction Reparameterisation

Instead of directly outputting the average velocity u, the u-head outputs a denoised data estimate $\hat{x}$ (x-prediction, following pMF [3]). The average velocity is derived via:

$$\hat{x} = f_\theta(z_t, r, t) \ \in \mathbb{R}^{C\times D\times H\times W}, \qquad u_\theta = \frac{z_t - \hat{x}}{\max(t,\, t_{\min})} \ \in \mathbb{R}^{C\times D\times H\times W} \tag{13}$$

where $t_{\min} = 0.05$ is a clamping threshold that prevents the $1/t$ singularity near $t = 0$. The division is element-wise after broadcasting t to shape $(B, 1, 1, 1, 1)$.

$\hat{x}, z_t, u_\theta$: each $(B, 4, 48, 48, 48)$. t: $(B,)$, clamped to $[t_{\min}, \infty)$, broadcast to $(B, 1, 1, 1, 1)$.
Code: `meanflow_loss.py:113-119`: `t_safe = t_.view(-1, *([1]*(z.ndim-1))).clamp(min=t_min)`; `return (z - x_hat) / t_safe`.

Justification (manifold hypothesis). The x-prediction target lies on the data manifold, which has low intrinsic dimensionality. The velocity u spans a higher-dimensional space. For architectures with a bottleneck (UNet encoder-decoder), predicting a low-dimensional target ($\hat{x}$) is easier than predicting a high-dimensional one (u).

Quantitative criterion (pMF Table 2): x-prediction dominates when $d_{\text{input}}/d_{\text{bottleneck}} > 1$. For our 3D UNet:

$$\frac{d_{\text{input}}}{d_{\text{bottleneck}}} = \frac{C \times D \times H \times W}{\text{channels}[3] \times (D/8)^3} = \frac{4 \times 48^3}{512 \times 6^3} = \frac{442{,}368}{110{,}592} \approx 4 \tag{14}$$

This is firmly in the x-prediction regime.

At inference (1-NFE). With x-prediction, 1-step sampling simplifies to a single forward pass—the model directly outputs $\hat{x} = z_0$:

$$z_0 = f_\theta(\varepsilon,\ r{=}0,\ t{=}1) \ \in \mathbb{R}^{C\times D\times H\times W} \tag{15}$$

ε: $(B, 4, 48, 48, 48)$. $r = \mathbf{0}_B$: $(B,)$; $t = \mathbf{1}_B$: $(B,)$. Output: $(B, 4, 48, 48, 48)$.
Code: `one_step.py:37-44`: `output = model(noise, r, t)`; `return output` (x-pred: direct).

2.8 Latent Space Formulation

All of the above operates identically in latent space. The computational advantages are:

1. **JVP cost reduction:** Latent dimension $d = 4 \times 48^3 = 442{,}368$ vs pixel space $d = 192^3 = 7{,}077{,}888$—a **16× reduction** in JVP compute per iteration.
2. **Memory reduction:** The UNet operates on 48^3 feature maps vs 192^3—approximately **64× reduction** in activation memory per JVP pass.
3. **Training efficiency:** The latent space is pre-computed (Phase 1), so VAE encode/decode cost is amortised across all training epochs.

3 Architecture Design

3.1 Frozen MAISI VAE (Encoder-Decoder)

The MAISI VAE [4] is a 3D variational autoencoder with adversarial training, pre-trained on ∼55K CT+MRI volumes. We use it as a **frozen foundation model**—all parameters are locked, and it is never fine-tuned. Key properties are listed in Table 3.

Table 3: MAISI VAE properties.

Property	**Value**
Parameters	20,944,897 (∼21 M)
Encoder stages	3 levels of 2× strided 3D conv
Latent channels	4 (with KL regularisation)
Spatial compression	4× per axis: $(1, 192^3) \to (4, 48^3)$
Attention	None (all `attention_levels=false`)
Training losses	L_1 + LPIPS + PatchGAN + KL
Checkpoint format	Wrapped in `"unet_state_dict"` key
Scale factor	$s = 0.96240234375$
Memory optimisation	`num_splits` for chunk-based processing

The VAE encoding and decoding operations are:

$$z = s \cdot \mathrm{Enc}_\phi(x_{\mathrm{MRI}}) \ \in \mathbb{R}^{4\times48\times48\times48}, \qquad \hat{x}_{\mathrm{MRI}} = \mathrm{Dec}_\phi(z \,/\, s) \ \in \mathbb{R}^{1\times192\times192\times192} \tag{16}$$

x_{MRI}: $(B, 1, 192, 192, 192)$ — single-channel T1 MRI.
z: $(B, 4, 48, 48, 48)$ — latent (4× spatial compression per axis, $1 \to 4$ channels).
$s = 0.9624$ — scale factor extracted from `diff_unet_3d_rflow-mr.pt["scale_factor"]`.

Reconstruction quality (Phase 0, 20 IXI volumes): Mean SSIM = 0.9213, Mean PSNR = 30.86 dB.

Latent distribution quality. Encoding all 6,471 volumes (train + val + test) confirms the latent space is well-regularised (Table 4).

Table 4: Per-channel latent statistics (n = 6,471 encoded volumes).

Channel	**Mean**	**Std**	**Skewness**	**Kurtosis**
0	−0.053	0.970	+0.105	−0.123
1	−0.185	1.019	+0.002	−0.019
2	−0.051	0.970	+0.045	+0.099
3	+0.001	1.011	+0.069	+0.144

The distributions are near-Gaussian: skewness $|\gamma| < 0.11$, excess kurtosis $|\kappa| < 0.15$, and standard deviations cluster around 0.97–1.02. Cross-channel correlations are negligible (max off-diagonal $|r| = 0.046$), confirming the KL regularisation produces approximately independent, unit-variance channels.

3.2 MeanFlow UNet (Generative Model)

The MeanFlow UNet (`MAISIUNetWrapper`) uses the **same architecture** as the MAISI diffusion UNet but with random initialisation and custom dual-time conditioning. Key properties are listed in Table 5.

Table 5: MeanFlow UNet properties and tensor shapes through the encoder-decoder path.

Property	**Value**	**Feature map shape**
Backbone	MONAI `DiffusionModelUNet` (3D)	—
Total parameters	∼178 M	—
`conv_in`	Conv3d(4, 64, 3, pad= 1)	$(B, 64, 48, 48, 48)$
Down block 0 (ch= 64)	2 ResBlocks, no attention	$(B, 64, 24, 24, 24)$
Down block 1 (ch= 128)	2 ResBlocks, no attention	$(B, 128, 12, 12, 12)$
Down block 2 (ch= 256)	2 ResBlocks + Transformer	$(B, 256, 6, 6, 6)$
Down block 3 (ch= 512)	2 ResBlocks + Transformer	$(B, 512, 3, 3, 3)$
Middle block (ch= 512)	ResBlock + Attn + ResBlock	$(B, 512, 3, 3, 3)$
Up blocks (mirror down)	With skip connections	up to $(B, 64, 48, 48, 48)$
`unet.out` (u-head)	GN + SiLU + Conv3d(64, 4)	$(B, 4, 48, 48, 48)$
`v_out` (v-head)	ResBlock + GN + SiLU + Conv3d(64, 4)	$(B, 4, 48, 48, 48)$
Attention heads	32 channels/head at levels 2–3	—
GroupNorm groups	32	—
Flash attention	**Disabled** (`torch.func.jvp` compat.)	—
Grad. checkpointing	**Disabled** (exact JVP compat.)	—
Prediction type	x-prediction ($\hat{x}$)	—

Flash attention incompatibility. `torch.func.jvp` uses forward-mode AD, which requires the computation graph to be fully traceable. Flash attention's fused CUDA kernels are opaque to PyTorch's AD system. Disabling flash attention adds ∼10% latency but enables exact JVP.

Gradient checkpointing incompatibility. Gradient checkpointing re-executes forward passes during backward, which conflicts with the forward-mode AD tape maintained by `torch.func.jvp`.

In-place operation patching. `torch.func.jvp` cannot handle in-place tensor mutations. The function `patch_inplace_ops()` (line 133 of `maisi_unet.py`) recursively replaces `nn.SiLU(inplace=True)` and `nn.ReLU(inplace=True)` with out-of-place variants throughout the model.

3.3 Dual-Time Conditioning

MeanFlow requires conditioning on two time variables: t (current time) and r (interval lower bound). We implement three conditioning modes and select `t_h` based on ablation (Table 6).

Table 6: Conditioning modes. Selected mode in **bold**.

Mode	**Inputs**	**Embedding**	**Source**
`dual`	(r, t)	$\mathbf{e}_r + \mathbf{e}_t$ via separate MLPs	pMF
`h`	$h = t - r$	$\mathbf{e}_h$ via UNet MLP	iMF
`t_h`	$(t, h{=}t{-}r)$	$\mathbf{e}_t + \mathbf{e}_h$ **via two MLPs**	**MF Tab. 1c**

The `t_h` mode conditions on both the absolute time t and the interval width $h = t - r$. The original MeanFlow paper (Table 1c) reports FID 61.06 for (t, h) vs 63.13 for h-only on ImageNet.

Sinusoidal embedding. Given a scalar $\tau \in [0, 1]$ (either t or h), the sinusoidal positional embedding [14] is defined as:

$$\text{SinEmb}(\tau) \coloneqq \left[\cos(\omega_j \cdot \tau'),\ \sin(\omega_j \cdot \tau')\right]_{j=0}^{d_{\text{sin}}/2-1} \in \mathbb{R}^{d_{\text{sin}}} \tag{17}$$

where $\tau' = 1000 \cdot \tau$ (time scaling), $d_{\text{sin}} = 64$ (= `channels[0]`), and the frequencies are $\omega_j = \exp(-\log(10000) \cdot j/(d_{\text{sin}}/2))$ for $j = 0, \ldots, 31$.

SinEmb(τ): $(B, 64)$.
Code: `maisi_unet.py:48-58`, `get_timestep_embedding(timesteps * 1000, embedding_dim=64)`.

Time embedding MLP. Each sinusoidal embedding is projected through a 2-layer MLP:

$$\text{MLP} : \mathbb{R}^{d_{\text{sin}}} \to \mathbb{R}^{d_{\text{emb}}}, \qquad \mathbf{e} = W_2\,\text{SiLU}(W_1\,\text{SinEmb}(\tau) + \mathbf{b}_1) + \mathbf{b}_2 \tag{18}$$

where $W_1 \in \mathbb{R}^{d_{\text{emb}} \times d_{\text{sin}}}$, $W_2 \in \mathbb{R}^{d_{\text{emb}} \times d_{\text{emb}}}$, $d_{\text{sin}} = 64$, $d_{\text{emb}} = 256$.

The combined conditioning embedding (in `t_h` mode) is:

$$\mathbf{e} = \text{MLP}_t(\text{SinEmb}(t)) + \text{MLP}_h(\text{SinEmb}(h)) \in \mathbb{R}^{d_{\text{emb}}} \tag{19}$$

SinEmb(t), SinEmb(h): each $(B, 64)$.
MLP_t output, MLP_h output: each $(B, 256)$.
$\mathbf{e}$: $(B, 256)$ — injected into every ResBlock and Transformer block.
Code: `maisi_unet.py:448-460`: `emb = t_emb + h_emb`.
MLP_t: `self.unet.time_embed` (MONAI built-in); MLP_h: `self.h_embed` (custom).

3.4 v-Head (Auxiliary Tangent Predictor)

The v-head is a lightweight auxiliary output path branching from the UNet's final feature map $\mathbf{h}$ (before the main output convolution):

```
Shared Backbone (B, 64, 48, 48, 48)
  |-- u-head: UNet out conv  --> (B, 4, 48, 48, 48)  [main]
  |-- v-head: ResBlock -> GN -> SiLU -> Conv3d --> (B, 4, 48, 48, 48)
```

The v-head ResBlock (`_VHeadResBlock`, `maisi_unet.py:205-235`) has the following structure:

$$\text{ResBlock}(\mathbf{h}) = \mathbf{h} + \underbrace{\text{Conv}_{3\times3\times3}^{64\to64}(\text{SiLU}(\text{GN}_{32}(\text{Conv}_{3\times3\times3}^{64\to64}(\text{SiLU}(\text{GN}_{32}(\mathbf{h}))))))}_{\text{zero-initialised second conv}} \tag{20}$$

$\mathbf{h} \in \mathbb{R}^{B\times64\times48\times48\times48}$. GN_{32}: GroupNorm with 32 groups over 64 channels.
Conv weights: $\mathbb{R}^{64\times64\times3\times3\times3}$ (= 110,592 params each). Second conv zero-initialised.
Total v-head: ∼228 K params (< 0.13% of 178 M backbone).

Table 7: v-Head properties.

Property	**Value**
ResBlocks	1
Parameters	∼228 K (<0.13% of total 178 M)
Initialisation	Final Conv3d zero-initialised (weights and bias)
Inference cost	Zero (output discarded at sampling)

Zero initialisation rationale. The v-head starts at identically zero output ($\hat{v} = \mathbf{0}$), so the u-head gradient signal is unperturbed at initialisation. Our training confirms: $\cos(\hat{v}, v_c)$ rises from 0.17 (epoch 0) to 0.39 (epoch 50) to 0.44 (epoch 689).

4 Loss Function and Training Objective

4.1 Per-Channel L_p Loss

The base loss function computes a per-channel, spatially-summed L_p norm. For tensors $\hat{y}, y \in \mathbb{R}^{C\times D\times H\times W}$:

$$\ell_p(\hat{y},\, y) \coloneqq \sum_{c=1}^{C}\sum_{i=1}^{D}\sum_{j=1}^{H}\sum_{k=1}^{W}|\hat{y}_{c,i,j,k} - y_{c,i,j,k}|^p \;\in \mathbb{R}_{\geq 0} \tag{21}$$

$\hat{y}, y \in \mathbb{R}^{B\times 4\times 48\times 48\times 48}$.
Output with `reduction="none"`: per-sample scalar $(B,)$—summed over all $C\times D\times H\times W = 4\times 48^3 = 442{,}368$ elements per sample.
Code: `lp_loss.py:34-48`: `diff = (pred - target).abs()`; `diff = diff.pow(p)`; `per_sample = diff.sum(dim=[1,2,3,4])`.

For the best model: p=2.0 (standard L_2), no per-channel weights. When optional channel weights $\mathbf{w} \in \mathbb{R}^C$ are provided:

$$\ell_p^{\mathbf{w}}(\hat{y}, y) = \sum_{c=1}^{C} w_c \sum_{i,j,k} |\hat{y}_{c,i,j,k} - y_{c,i,j,k}|^p \tag{22}$$

This extends the SLIM-Diff per-channel loss [6] from pixel-space DDPM to latent-space MeanFlow.

4.2 Combined iMF Dual-Head Loss

The training loss has two independently-weighted components:

$$\boxed{\mathcal{L} = \frac{1}{B}\sum_{b=1}^{B}\left[\frac{\ell_p^{(b)}(V_\theta,\, v_c)}{w_u^{(b)}} + \frac{\ell_p^{(b)}(\hat{v},\, v_c)}{w_v^{(b)}}\right]} \in \mathbb{R} \tag{23}$$

where the superscript (b) denotes the b-th sample in the batch, and:

- $\ell_p^{(b)}(V_\theta, v_c)$ is the **compound velocity loss** (u-head): enforces MeanFlow self-consistency.
- $\ell_p^{(b)}(\hat{v}, v_c)$ is the **tangent supervision loss** (v-head): directly supervises the auxiliary head.
- $w_u^{(b)}, w_v^{(b)}$ are per-sample adaptive weights (defined below).

Both losses target the same conditional velocity $v_c = \varepsilon - z_0 \in \mathbb{R}^{C\times D\times H\times W}$.

$V_\theta, \hat{v}, v_c$: each $(B, 4, 48, 48, 48)$.
$\ell_p^{(b)}(\cdot,\cdot)$: scalar per sample, so the full vector is $(B,)$.
$\mathcal{L}$: scalar (mean over batch).
Code: `meanflow_loss.py:342-356`: `raw_loss_u = lp_loss(V, v_c, ...)`; `raw_loss_v = lp_loss(v, v_c, ...)`; `loss = (loss_u + loss_v).mean()`.

4.3 Adaptive Weighting

Each loss component is independently normalised by its own magnitude to prevent either head from dominating the gradient. For a generic per-sample raw loss $\ell^{(b)} \in \mathbb{R}_{\geq 0}$, the adaptive weight and weighted loss are:

$$w^{(b)} \coloneqq \left(\mathrm{sg}[\ell^{(b)}] + \varepsilon_{\text{norm}}\right)^{p_{\text{norm}}} \;\in \mathbb{R}_{>0} \tag{24}$$

$$\ell^{(b)}_{\text{weighted}} \coloneqq \frac{\ell^{(b)}}{w^{(b)}} \;\in \mathbb{R}_{\geq 0} \tag{25}$$

where sg[·] is the stop-gradient operator (Eq. 1) ensuring the weight does not contribute gradients, $\varepsilon_{\text{norm}} = 1.0$, and $p_{\text{norm}} = 1.0$. Substituting these defaults:

$$w^{(b)} = \text{sg}[\ell^{(b)}] + 1.0, \qquad \ell^{(b)}_{\text{weighted}} = \frac{\ell^{(b)}}{\text{sg}[\ell^{(b)}] + 1.0} \tag{26}$$

$\ell^{(b)}$: per-sample scalar; full batch: $(B,)$.
$w^{(b)}$: per-sample scalar; full batch: $(B,)$. Computed with `.detach()` on ℓ.
$\ell^{(b)}_{\text{weighted}}$: per-sample scalar; full batch: $(B,)$.
Code: `meanflow_loss.py:347-351`: `adp_weight_u = (raw_loss_u.detach() + norm_eps) ** norm_p`; `loss_u = raw_loss_u / adp_weight_u`. Applied **independently** to u-head and v-head.

This is a form of **logarithmic loss normalisation**: the effective loss behaves as $\ell_{\text{w}} \approx \ell/(\ell+1)$, which is bounded in $[0, 1)$ and varies slowly with raw loss magnitude:

$$\lim_{\ell \to 0} \frac{\ell}{\ell+1} = 0, \qquad \lim_{\ell \to \infty} \frac{\ell}{\ell+1} = 1 \tag{27}$$

At convergence (epoch 388, best FID), the loss decomposition shows this balancing in action (Table 8).

Table 8: Loss decomposition at best FID (epoch 388).

Component	**Raw Loss**	**Share**
FM loss ($r = t$, flow matching)	715,432	10.3%
MF loss ($r < t$, self-consistency)	6,254,997	89.7%
v-head loss	714,646	—
u-head loss (compound V)	3,485,215	—

The MF loss is ∼9× larger than the FM loss, which is expected: the compound velocity V must also capture the JVP correction term, a harder target than the direct velocity at $r{=}t$. The adaptive weighting equalises their gradient contributions to ∼1.0 each.

Why $\varepsilon_{\text{norm}}$=1.0? Through Phase 4c debugging, we discovered that $\varepsilon_{\text{norm}}$=0.01 caused catastrophic gradient amplification for samples with small raw loss. The weight $1/(\ell + 0.01)$ can reach 100×, creating a positive feedback loop.

Why p_{norm}=1.0? With p_{norm}=0.5, the weight becomes $w = \sqrt{\ell + 1.0}$, which under-normalises large losses, leading to a 1000× gradient explosion (Phase 4e).

4.4 JVP Strategies

Two strategies are implemented for computing the directional derivative in Eq. (9).

Strategy 1: Exact JVP (`torch.func.jvp`). Computes the exact directional derivative via forward-mode automatic differentiation:

$$(u_\theta,\ \tfrac{du}{dt},\ \hat{v}) = \texttt{torch.func.jvp}\Big(\underbrace{f_{\text{dual}}}_{\mathbb{R}^d\times\mathbb{R}\times\mathbb{R}\to\mathbb{R}^d\times\mathbb{R}^d},\ \underbrace{(z_t,\, t,\, r)}_{\text{primals}},\ \underbrace{(\tilde{v},\, \mathbf{1},\, \mathbf{0})}_{\text{tangents}},\ \texttt{has\_aux=True}\Big) \tag{28}$$

Primals: $z_t : (B, 4, 48, 48, 48)$; $t : (B,)$; $r : (B,)$.
Tangents: $\tilde{v} : (B, 4, 48, 48, 48)$; $\mathbf{1}_B : (B,)$; $\mathbf{0}_B : (B,)$.
Returns: $u_\theta : (B, 4, 48, 48, 48)$; $du/dt : (B, 4, 48, 48, 48)$; $\hat{v} : (B, 4, 48, 48, 48)$.
`has_aux=True` captures $\hat{v}$ without computing its JVP (zero overhead).
Compound: $V = u + (t - r) \cdot \text{sg}[du/dt]$: $(B, 4, 48, 48, 48)$.
Code: `jvp_strategies.py:109-119`. Memory: ∼20 GB activations per sample at B=2 on A100-40GB.

Here $f_{\text{dual}}(z,t,r) := (u_\theta(z,t,r),\ \hat{v}(z,t,r))$ returns both the average velocity (through which the JVP is computed) and the v-head output (captured as auxiliary, no JVP). The function `_u_with_v_aux` wraps the dual model output so that `has_aux=True` passes the v-head through unchanged.

Strategy 2: Finite Difference JVP (FD-JVP). Approximates the directional derivative using a perturbation of step size $\delta = 10^{-3}$:

$$\left.\frac{du_\theta}{dt}\right|_{\tilde{v}} \approx \frac{u_\theta^{(\text{fp32})}(z_t + \delta\,\tilde{v},\ t+\delta,\ r) - u_\theta^{(\text{fp32})}(z_t,\ t,\ r)}{\delta} \in \mathbb{R}^{C\times D\times H\times W} \tag{29}$$

Perturbed inputs: $z_t + \delta\tilde{v}$: $(B, 4, 48, 48, 48)$; $t+\delta$: $(B,)$.
Both u terms cast to fp32 before subtraction (avoids bf16 catastrophic cancellation).
Perturbed pass runs under `torch.no_grad()` (since du/dt is always detached).
Code: `jvp_strategies.py:156-168`.

Critical incompatibility (Phase 4f). x-prediction + FD-JVP is **numerically unstable**. The x-to-u conversion $u = (z_t - \hat{x})/t$ has a $1/t$ singularity. When FD-JVP computes the difference quotient, the result involves:

$$\left.\frac{du}{dt}\right|_{\text{FD}} = \frac{u(t+\delta) - u(t)}{\delta} \sim \frac{O(1/t)}{\delta} = O\left(\frac{1}{t^2\,\delta}\right) \to \infty \quad \text{as } t \to t_{\min} \tag{30}$$

With exact JVP, the $1/t$ factor is analytically differentiated via the chain rule, yielding stable gradients.

Rule: x-pred + exact JVP = stable | u-pred + FD-JVP = stable | x-pred + FD-JVP = explosion

5 Data Pipeline

5.1 Dataset

We use 8 datasets from FOMO-60K, a large-scale preprocessed brain MRI collection (Table 9).

Table 9: Dataset composition from 8 FOMO-60K subsets.

Dataset	Train Subj.	Train Scans	Val Scans	Test Scans
PT001_OASIS1	352	1,414	170	96
PT002_OASIS2	71	682	82	37
PT005_IXI	494	494	58	29
PT007_NIMH	212	428	48	23
PT008_DLBS	394	807	109	51
PT011_MBSR	125	295	36	16
PT012_UCLA	106	106	13	6
PT015_NKI	722	1,245	158	68
Total	**2,476**	**5,471**	**674**	**326**

Many subjects have multiple sessions (longitudinal scans), yielding 5,471 training scans from 2,476 subjects—approximately 4× more data than the initially planned ∼1,172 scans from 3 datasets. All volumes are T1-weighted, skull-stripped, RAS-oriented, and co-registered. The split is stratified by dataset with a fixed seed of 42 for reproducibility.

5.2 Preprocessing Pipeline

1. `LoadImaged` (MONAI)
2. `EnsureChannelFirstd`
3. `Spacingd`(pixdim=1.0 mm isotropic, bilinear)
4. `ScaleIntensityRangePercentilesd`(0%–99.5% $\rightarrow [0, 1]$)
5. `CropForegroundd`(margin=4) — brain-centred crop
6. `ResizeWithPadOrCropd`$(192, 192, 192)$ — pad to target
7. `EnsureTyped`(dtype=float32)

Resolution choice (192^3 at 1.0 mm isotropic). Selected after quantitative analysis of brain extent across 30 FOMO-60K subjects. Brain AP extent reaches 193 mm; 128^3 and 160^3 both clip frontal/occipital cortex. `CropForegroundd` centres the crop on brain tissue, achieving 100% brain coverage for all subjects.

Latent shape: $192/4 = 48$ per spatial axis, giving latents of shape $(4, 48, 48, 48)$.

5.3 Latent Pre-Computation (Phase 1)

All training volumes are pre-encoded through the frozen MAISI VAE and stored in HDF5 shards. A total of 6,471 volumes were encoded (∼11.5 GB storage).

Latent statistics are computed online during encoding via `LatentStatsAccumulator` (sum-of-powers method in float64): per-channel mean, std, skewness, kurtosis, min, max, and a 4×4 cross-channel Pearson correlation matrix.

5.4 Latent Augmentation

Three augmentation transforms operate directly on latents during training (Table 10).

Table 10: Latent augmentation transforms.

Transform	Prob.	Params	Rationale
Flip depth axis	0.5	L–R in RAS	Bilateral symmetry
Gaussian noise	0.2	5% of per-ch. std	Encoding noise
Intensity scale	0.2	±5%	Scanner variation

Approximately 68% of training samples are augmented per epoch (∼3,720 out of 5,471).

6 Training Protocol

6.1 Algorithm (One Training Step)

Given a mini-batch of B pre-computed latents and the dual-head model f_θ (with u-head and v-head), one training step proceeds as follows. All tensors are annotated with their shapes.

1. **Load data:** $z_0 \in \mathbb{R}^{B\times4\times48\times48\times48}$ (normalised latent from HDF5).
2. **Sample noise:** $\varepsilon \sim \mathcal{N}(\mathbf{0}, I_d) \in \mathbb{R}^{B\times4\times48\times48\times48}$.
3. **Sample times:** $t, r \sim \text{LogitNormal}(\mu{=}-0.4, \sigma{=}1.0) \in \mathbb{R}^B$, clamped to [0.001, 1.0], with $t \geq r$. For half the batch ($\lfloor B/2 \rfloor$ samples): set $r = t$ (FM samples).
4. **Interpolate:** $z_t = (1 - \bar{t}) \odot z_0 + \bar{t} \odot \varepsilon \in \mathbb{R}^{B\times4\times48\times48\times48}$, where $\bar{t} = t.\text{view}(B, 1, 1, 1, 1)$.
5. **Target:** $v_c = \varepsilon - z_0 \in \mathbb{R}^{B\times4\times48\times48\times48}$.

6. **Tangent (no grad):** $(\_, \tilde{v}) = f_\theta(z_t,\, r{=}t,\, t)$; $\tilde{v} \in \mathbb{R}^{B\times4\times48\times48\times48}$ is the v-head output at $r{=}t$ (instantaneous). Computed under `torch.no_grad()`.
7. **JVP (exact, dual):** $(u_\theta,\ du/dt,\ \hat{v}) = \texttt{jvp}(f_{\text{dual}},\ (z_t, t, r),\ (\tilde{v}, \mathbf{1}, \mathbf{0}),\ \texttt{has\_aux})$.
   - $u_\theta \in \mathbb{R}^{B\times4\times48\times48\times48}$ (average velocity, derived from $\hat{x}$ via Eq. 13).
   - $du/dt \in \mathbb{R}^{B\times4\times48\times48\times48}$ (JVP output).
   - $\hat{v} \in \mathbb{R}^{B\times4\times48\times48\times48}$ (v-head output, auxiliary—no JVP through it).
8. **Compound velocity:** $V_\theta = u_\theta + (t-r).\text{view}(B,1,1,1,1) \cdot \text{sg}[du/dt] \in \mathbb{R}^{B\times4\times48\times48\times48}$.
9. **Per-sample raw losses:** $\ell_u^{(b)} = \|V_\theta^{(b)} - v_c^{(b)}\|_2^2 \in \mathbb{R}$, $\ell_v^{(b)} = \|\hat{v}^{(b)} - v_c^{(b)}\|_2^2 \in \mathbb{R}$; vectors $\boldsymbol{\ell}_u, \boldsymbol{\ell}_v \in \mathbb{R}^B$.
10. **Adaptive weighting** (Eq. 26): $\mathcal{L} = \frac{1}{B}\sum_{b=1}^{B}\left[\frac{\ell_u^{(b)}}{\text{sg}[\ell_u^{(b)}]+1} + \frac{\ell_v^{(b)}}{\text{sg}[\ell_v^{(b)}]+1}\right] \in \mathbb{R}$.
11. **Backward + clip:** $\nabla_\theta \mathcal{L}$; clip $\|\nabla\|_2 \le 1.0$.
12. **Optimiser step + EMA update:** $\theta \leftarrow \theta - \eta\,\nabla_\theta\mathcal{L}$; $\theta_{\text{EMA}} \leftarrow \beta\,\theta_{\text{EMA}} + (1-\beta)\,\theta$.

6.2 Time Sampling

Times are drawn from a logit-normal distribution. Given a standard normal sample $\xi \sim \mathcal{N}(0,1)$, the logit-normal transform is:

$$t = \sigma(\mu + \sigma_t\,\xi) = \frac{1}{1+e^{-(\mu+\sigma_t\xi)}} \in (0,1), \qquad t \leftarrow \max(t,\ t_{\min}) \tag{31}$$

where $\sigma(\cdot)$ is the sigmoid function, $\mu = -0.4$ (biases toward lower t, *i.e.*, near data), $\sigma_t = 1.0$, and $t_{\min} = 0.001$.

ξ: $(B,)$; t: $(B,)$ after clamp.
Both t and r are sampled independently from the same distribution, then swapped so $t \ge r$:
$t \leftarrow \max(t_{\text{raw}}, r_{\text{raw}})$; $r \leftarrow \min(t_{\text{raw}}, r_{\text{raw}})$.
Code: `time_sampler.py:62-73`.

data_proportion $= 0.5$**.** For the first $\lfloor B/2 \rfloor$ samples in the batch, r is set equal to t:

$$r^{(b)} = \begin{cases} t^{(b)} & \text{if } b < \lfloor B/2 \rfloor \quad \text{(FM sample: } h=0\text{, JVP vanishes)} \\ r_{\text{raw}}^{(b)} & \text{otherwise} \quad \text{(MF sample: } h>0\text{, self-consistency enforced)} \end{cases} \tag{32}$$

Boundary-aware injection (v2). The logit-normal distribution assigns negligible mass to the 1-NFE operating point: $P(t > 0.95) = 1 - \Phi(3.35) \approx 0.04\%$. To provide direct training signal at this boundary, we replace the base distribution with a mixture:

$$p(t,r) = (1-\alpha)\,p_{\text{LN}}(t,r) + \alpha\,\mathcal{U}([1{-}\delta,\ 1] \times [t_{\min},\ \delta]), \qquad \alpha = 0.1,\ \delta = 0.05 \tag{33}$$

where p_{LN} is the logit-normal process above. The boundary fraction $\alpha = 10\%$ is deliberately generous: the adaptive weight $w = \ell/(\ell{+}\varepsilon)$ partially damps high-loss boundary samples (gradient $\propto 1/\ell$ for large ℓ), so surplus injection overcomes this attenuation. At $\alpha = 0.1$ with $\delta = 0.05$, at least 8% of every batch probes the region $(t > 0.95,\ r < 0.05)$ needed for 1-NFE generation.

6.3 Optimiser and Schedule

Hyperparameters are listed in Table 11.

Warmup (v2). The v1 run used no warmup, which resulted in 90.5% of gradient updates being clipped at epoch 0 (gradient norm exceeding the clip threshold of 1.0 on nearly every step). By epoch 46 (∼2,000 steps) the clip fraction dropped to 21.4%. In v2 we introduce a linear warmup over 1,000 steps (∼24 epochs), covering this unstable regime and following the recommendation of Goyal *et al.* [16] for large effective batch sizes ($B_{\text{eff}} = 132$).

Table 11: Optimiser hyperparameters.

Hyperparameter	**Value**	**Justification**
Optimiser	AdamW	Standard for diffusion/flow models
Learning rate	$\eta = 10^{-4}$	MeanFlow reference default
Weight decay	0.0	No regularisation with adaptive loss
Betas	$(\beta_1, \beta_2) = (0.9, 0.95)$	β_2=0.95: faster momentum adaptation
LR schedule	Cosine decay	$10^{-4} \to 0$ over total steps
Warmup	1,000 steps (v2)	Linear warmup; 90.5% grad clipping at step 0
Gradient clip	$\|\nabla\|_2 \leq 1.0$	Prevents spike propagation
Mixed precision	bf16	Standard for A100

6.4 Compute Budget

Table 12: Compute budget and actual training cost.

Property	**v1**	**v2 (planned)**
Hardware	6× A100-SXM4-40GB (DDP)	
Per-GPU batch	$B_{\text{gpu}} = 2$	
Gradient accumulation	11 steps	
Effective batch	$B_{\text{eff}} = 2 \times 6 \times 11 = 132$	
Optimiser steps/epoch	42	
Max epochs	1,500	3,000
Planned optimiser steps	63,000	126,000
Actual optimiser steps	28,980 (early stop)	—
Early-stop patience	10 FID evals	30 FID evals
FID eval frequency	every 30 epochs	every 10 epochs
Warmup steps	0	1,000
Mean epoch time	∼268 s	
Total wall time (v1)	**∼51.4 hours**	

6.5 EMA (Exponential Moving Average)

An EMA model maintains shadow parameters θ_{EMA} that are a running weighted average of the online parameters θ. The update rule, applied after each optimiser step, is:

$$\theta_{\text{EMA}}^{(k+1)} := \beta\, \theta_{\text{EMA}}^{(k)} + (1 - \beta)\, \theta^{(k+1)} \tag{34}$$

where $\beta = 0.9999$ is the decay rate (half-life $\approx$ 6,931 steps), k is the optimiser step index, $\theta^{(k+1)}$ are the parameters after the k-th optimiser step, and $\theta_{\text{EMA}}^{(0)} = \theta^{(0)}$ (initialised to model weights).

$\theta, \theta_{\text{EMA}}$: all trainable parameters (∼178 M scalars).
Code: `ema.py:50`: `shadow[name].mul_(decay).add_(param.data, alpha=1-decay)`.
Applied per-parameter: each shadow tensor has the same shape as its corresponding parameter.

All FID evaluations during training use EMA weights. The best model checkpoint is selected by FID computed from EMA-weight samples.

6.6 Divergence Monitoring

An EMA-smoothed raw loss monitor (decay 0.99, half-life ∼69 steps) tracks training stability. Warnings fire at 3× and 5× the minimum EMA loss, with a grace period of 1000 steps. Early stopping is FID-based via `EvaluationCallback` with patience = 10.

7 Sampling and Generation

7.1 One-Step Sampling (1-NFE)

With x-prediction, 1-NFE sampling is a single forward pass:

$$\varepsilon \sim \mathcal{N}(\mathbf{0},\, I_d) \;\in \mathbb{R}^{B\times C\times D\times H\times W}, \qquad z_0 = f_\theta(\varepsilon,\; r{=}\mathbf{0}_B,\; t{=}\mathbf{1}_B) \;\in \mathbb{R}^{B\times C\times D\times H\times W} \tag{35}$$

ε, z_0: $(B, 4, 48, 48, 48)$. $r = \mathbf{0}_B$, $t = \mathbf{1}_B$: each $(B,)$.
The v-head output is discarded: `if isinstance(output, tuple): output = output[0]`.
Code: `one_step.py`.

Norm correction (v2, post-training). The compound velocity norm ratio $\|V_\theta\|/\|v_c\|$ at epoch 689 is $1092/941 \approx 1.16$, indicating a systematic 16% overshoot. At inference this translates to overshooting the data manifold. A scalar correction factor γ is applied:

$$z_0 = \varepsilon - \frac{u_\theta(\varepsilon,\, 0,\, 1)}{\gamma}, \qquad u_\theta = \frac{\varepsilon - \hat{x}_\theta}{1} \tag{36}$$

where γ is calibrated post-training on a grid $\{1.0, 1.05, 1.10, 1.15, 1.20\}$ by minimising FID-3D at NFE=1. Default: $\gamma = 1.0$ (no correction). This is analogous to the guidance rescale used in classifier-free guidance [17].

7.2 Multi-Step Euler Sampling

For comparison, multi-step Euler sampling divides $[1, 0]$ into K uniform steps with timesteps $t_i = 1 - i/K$ for $i = 0, \ldots, K$:

$$\Delta t_i \coloneqq t_i - t_{i+1} = \frac{1}{K}, \qquad t_i, t_{i+1} \in \{1,\; 1{-}\tfrac{1}{K},\; \ldots,\; 0\} \tag{37}$$

At each step, the model predicts $\hat{x}$, which is converted to velocity and used for an Euler update:

$$\hat{x}_i = f_\theta(z^{(i)},\; r{=}t_{i+1},\; t{=}t_i) \;\in \mathbb{R}^{C\times D\times H\times W}, \qquad u_i = \frac{z^{(i)} - \hat{x}_i}{\max(t_i,\, 0.05)}, \qquad z^{(i+1)} = z^{(i)} - \Delta t_i \cdot u_i \tag{38}$$

$z^{(i)}, \hat{x}_i, u_i$: each $(B, 4, 48, 48, 48)$. $z^{(0)} = \varepsilon$; $z^{(K)} = z_0$ (final sample).
Code: `multi_step.py`; NFE levels tested: $K \in \{1, 2, 5, 10, 25, 50\}$.

When norm correction is enabled (Section 7.1), the Euler step becomes $z^{(i+1)} = z^{(i)} - \Delta t_i \cdot u_i/\gamma$.

7.3 Full Generation Pipeline (Phase 5)

1. Pre-generate shared noise $\varepsilon \in \mathbb{R}^{N\times 4\times 48\times 48\times 48}$ (for NFE-consistency analysis).
2. For each NFE level: generate latents, store in HDF5 (latents, seeds, timing).
3. Denormalise: $z_0 = \hat{z} \cdot \sigma_{\text{latent}} + \mu_{\text{latent}}$, where $\mu_{\text{latent}}, \sigma_{\text{latent}} \in \mathbb{R}^{1\times 4\times 1\times 1\times 1}$.
4. Decode: $\hat{x}_{\text{MRI}} = \text{Dec}_\phi(z_0 \,/\, s) \in \mathbb{R}^{B\times 1\times 192\times 192\times 192}$.
5. Clamp to $[0, 1]$; store decoded volumes in HDF5.

Shared noise protocol. The same noise tensor ε is used across all NFE levels for a given sample index, enabling direct comparison of how additional steps refine the output.

8 Experimental Results

8.1 Toy Validation (Phase 2): MeanFlow on Flat Torus

Before scaling to brain MRI, MeanFlow was validated on a flat torus in $\mathbb{R}^4$ (Table 13).

Table 13: Phase 2 toy validation results.

Metric	Result	Target
1-NFE torus distance	0.0892	<0.1
5-step torus distance	0.0271	<0.05
Angular KS p-value (θ_1)	0.374	>0.01
Angular KS p-value (θ_2)	0.575	>0.01
x-pred vs u-pred loss ratio	<1.2	<1.5

This confirmed: (i) the MeanFlow implementation correctly learns 1-step generation, (ii) the angular distribution is statistically indistinguishable from truth (KS test), and (iii) both x-prediction and u-prediction converge.

8.2 Main Training (Phase 4): Best Model

The best model uses x-prediction + exact JVP + (t, h) conditioning + v-head (1 ResBlock) + L_2 loss (Table 14).

Table 14: Best model configuration.

Property	Value
Prediction type	x-prediction
JVP strategy	Exact (`torch.func.jvp`)
Conditioning	t_h (condition on t and $h = t-r$)
v-head	1 ResBlock ($\sim$228 K params, $<0.13\%$)
Loss norm	L_2 (p=2) with independent adaptive weighting
Best FID epoch	**388** (step 16,338)
Early-stopped at	Epoch 690 (patience=10, 46% of planned 1,500)

8.3 FID Progression

FID is computed as a 2.5D metric using RadImageNet ResNet-50 features on central axial, coronal, and sagittal slices (Table 15).

FID drops rapidly from 46.76 to $\sim$14 in the first 88 epochs, then gradually improves to **11.67** at epoch 388, after which it plateaus. Training continued for 302 more epochs without improvement before early stopping triggered.

8.4 Training Dynamics

Table 16 shows key training metrics at milestone epochs.

Key observations:

Table 15: 2.5D FID progression during training (selected checkpoints).

Epoch	**Step**	FID_{avg}	FID_{xy}	FID_{yz}	**Notes**
9	420	46.76	45.12	46.68	First evaluation
28	1,218	28.40	17.25	43.63	Rapid initial drop
88	3,738	14.61	14.08	16.25	Approaching plateau
178	7,518	14.06	13.79	15.46	
268	11,298	12.94	12.77	14.06	
388	**16,338**	**11.67**	**11.85**	**12.09**	**Best FID**
628	26,418	11.92	12.03	12.33	
688	28,938	11.88	12.14	12.08	Final evaluation

Table 16: Training dynamics at milestone epochs.

Epoch	**Raw Loss**	$\cos(V, v_c)$	$\cos(\hat{v}, v_c)$	**Rel. Err.**	$\|\nabla\|$	**LR**
0	2,788,299	0.083	0.170	1.353	1.384	1.0×10^{-4}
50	5,316,270	0.240	0.388	1.759	0.953	$\sim9.9\times10^{-5}$
100	6,275,983	0.274	0.412	1.778	0.910	$\sim9.7\times10^{-5}$
388	4,724,751	0.295	0.429	1.645	1.180	8.5×10^{-5}
689	4,327,238	0.307	0.436	1.516	0.610	5.6×10^{-5}

1. **Raw loss is not a good proxy for generation quality.** The raw loss is dominated by the MF term ($\ell_{\text{MF}} \approx 6.5\times10^6$ vs $\ell_{\text{FM}} \approx 7\times10^5$) and does not monotonically decrease. FID (computed from EMA-weight samples) is the correct early-stopping metric.
2. **Cosine alignment improves steadily.** $\cos(V, v_c)$ increases from 0.083 to 0.307; the v-head alignment converges faster (0.170 → 0.436), confirming it provides good tangents from early training.
3. **Gradient norm decreases dramatically.** From 1.384 to 0.610; clipping fraction drops ∼90%→∼5%, indicating stable dynamics by epoch ∼300.

8.5 Generated Sample Statistics

The model's output $\hat{x}$ statistics evolve during training (Table 17).

Table 17: Statistics of generated $\hat{x} \in \mathbb{R}^{4\times48\times48\times48}$ at different training stages.

Epoch	$\mathbb{E}[\hat{x}]$	$\text{std}[\hat{x}]$	$\min[\hat{x}]$	$\max[\hat{x}]$
0	+0.004	0.243	−2.06	3.41
100	−0.001	0.737	−5.88	8.56
388	−0.001	0.753	−5.84	8.69
689	−0.001	0.781	−6.22	9.38

The standard deviation stabilises around 0.75–0.78, approximately 75% of the true latent std (∼0.97–1.02). This **variance under-estimation** is the primary quality bottleneck: generated samples have slightly less contrast than real ones.

8.6 NFE Quality Analysis

Visual inspection of decoded samples reveals a clear quality hierarchy (Table 18).

Table 18: Visual quality at different NFE levels.

NFE	Quality	Key Features
1	Recognisable, blurry	Global structure correct, low contrast
2	Substantially sharper	Cortical sulci visible
5	Sharp, plausible	Clear grey-white contrast
10	Marginally crisper	Diminishing returns beyond 5

NFE-consistency metrics (shared noise, converged model): cosine similarity NFE=1 vs NFE=2: ∼0.96; vs NFE=5: ∼0.91; vs NFE=10: ∼0.90.

The 1-NFE output captures global anatomy correctly but lacks high-frequency detail—a known MeanFlow limitation. NFE=2 provides substantial improvement at only 2× cost.

8.7 Spectral Analysis

The radially-averaged power spectrum evolves from flat (noise-like at epoch ∼100) to the characteristic $1/f$ brain spectrum by epoch 600. All 4 channels show proper low-frequency dominance with high-frequency rolloff.

8.8 SWD (Sliced Wasserstein Distance)

SWD improves rapidly in early training (best = 1.779 at epoch 9) but then **diverges from FID**—SWD increases monotonically after epoch ∼50 (final: 2.555) while FID continues to decrease. SWD is **not a reliable proxy** for perceptual quality in this setting.

8.9 Key Ablation Insights

x-pred vs u-pred. x-prediction with exact JVP trained stably through 690 epochs (best FID at 388). This supports the pMF manifold hypothesis: $d_{\text{input}}/d_{\text{bottleneck}} \approx 4$, firmly in the x-prediction regime.

x-pred + FD-JVP instability. Exponential growth of JVP norms (10,000× within 50 epochs). Root cause (Eq. 30):

$$u = \frac{z_t - \hat{x}}{t} \implies \left.\frac{du}{dt}\right|_{\text{FD}} \approx \frac{u(t+\delta) - u(t)}{\delta} \sim \frac{O(1/t)}{\delta} = O\left(\frac{1}{t^2\delta}\right) \tag{39}$$

Conditioning mode. (t, h) selected per MeanFlow Table 1c: FID 61.06 for (t, h) vs 63.13 for h-only.

Training stability fixes (Phase 4c–4e):

- β_2: 0.999 → 0.95 (prevents stale gradients).
- Warmup: 5000 → 0 (destabilises MF loss).
- $\varepsilon_{\text{norm}}$: 0.01 → 1.0 (prevents runaway amplification).
- p_{norm}: 0.5 → 1.0 (sub-linear normalisation → gradient explosion).
- data_proportion: stabilised at 0.5 (50/50 FM/MF split).

9 Baseline Comparison

9.1 MOTFM Baseline

Medical Optimal Transport Flow Matching (MOTFM) [7] is our primary baseline: a conditional flow matching model operating in pixel space with an ODE solver at inference. Unlike NeuroMF, MOTFM requires multiple integration steps (typically 10 Euler steps) and works directly on voxel-space volumes rather than in a compressed latent space.

9.1.1 Architecture

MOTFM uses a `DiffusionModelUNet` from MONAI-Generative with 4 resolution levels: channel widths $[32, 64, 128, 256]$, 2 residual blocks per level, attention only at the coarsest level, and flash attention enabled. The model is unconditional (no mask or class conditioning) for fair comparison with NeuroMF on the same dataset.

9.1.2 Step-Matched Training Budget

To ensure a fair comparison, we match the total number of *optimiser steps* rather than epochs, since our dataset is 7.3× larger than the one used in the original MOTFM paper.

Table 19: MOTFM training budget: paper vs. our step-matched setup.

Property	**Paper**	**Ours**
Dataset	MSD Brain (750 scans)	FOMO-60K (5,471 scans)
Resolution	192^3	192^3
Hardware	1× RTX 4090	4× A100-40GB (DDP)
Batch size (per GPU)	1	1
Gradient accumulation	1	1
Effective batch size	1	4
Steps per epoch	750	1,368
Epochs	200	110
Total optimiser steps	**150,000**	**∼150,480**
Learning rate	10^{-4}	10^{-4}
Precision	fp32	bf16-mixed
Solver	Euler, 10 steps	Euler, 10 steps
Est. wall time	—	∼69 h (2.9 days)
SLURM wall limit	—	4 days

The step-matching rationale is straightforward:

$$N_{\text{epochs}}^{\text{ours}} = \left\lceil \frac{N_{\text{paper}} \cdot E_{\text{paper}}}{\lceil S_{\text{ours}}/G \rceil} \right\rceil = \left\lceil \frac{750 \times 200}{\lceil 5471/4 \rceil} \right\rceil = \left\lceil \frac{150{,}000}{1{,}368} \right\rceil = 110 \tag{40}$$

where $S_{\text{ours}} = 5{,}471$ is our training set size and $G = 4$ is the GPU count. The effective batch size increases from 1 to 4 due to DDP data parallelism; this is standard practice and commonly accepted for fair comparison, as each GPU still processes one sample per step with no gradient accumulation.

Table 20: Architectural and methodological differences between NeuroMF and MOTFM.

Property	**NeuroMF**	**MOTFM**
Operating space	Latent (4×48^3)	Pixel (1×192^3)
NFE (inference)	1	10 (Euler)
Training paradigm	MeanFlow (iMF dual-head)	Flow matching (velocity)
VAE dependency	Frozen MAISI VAE	None
Model parameters	∼178M	TBD
Conditioning	Unconditional	Unconditional

9.1.3 Key Differences from NeuroMF

10 Evaluation Protocol

10.1 Image Quality Metrics

Table 21: Evaluation metrics suite.

Metric	**Description**	**Features**
2.5D FID	Slice-wise FID (ax/cor/sag)	RadImageNet ResNet-50
3D FID	Volumetric FID	Med3D ResNet-50
MMD	Max Mean Discrepancy	Gaussian kernel
Coverage	Real–synthetic NN fraction	Feature-space NN
Density	Synthetic near real manifold	Complementary to coverage
MS-SSIM	3-level Multi-Scale SSIM (3D)	MONAI `SSIMMetric`
PSNR	Peak Signal-to-Noise Ratio	Actual data range

10.2 Spectral Analysis

High-frequency energy ratio via 3D FFT:

$$\rho := \frac{\sum_{|\mathbf{k}|>k_0} |\mathcal{F}(x)(\mathbf{k})|^2}{\sum_{\mathbf{k}} |\mathcal{F}(x)(\mathbf{k})|^2} \in [0,1] \tag{41}$$

where $\mathcal{F}(x) \in \mathbb{C}^{D' \times H' \times W'}$ (D'=H'=W'=192) is the 3D discrete Fourier transform, $\mathbf{k} \in \mathbb{Z}^3$ is the frequency index, $|\mathbf{k}|$ is the radial frequency, and k_0 is the cutoff. Reported for real, VAE-reconstructed, and generated volumes.

10.3 Morphological Assessment

SynthSeg [13] segmentation on both real and synthetic volumes: regional volume correlation, distributional KL divergence, and SynthSeg Dice overlap (paired by nearest-neighbour in feature space).

10.4 NFE Consistency Analysis

Generated samples at NFE $\in \{1, 2, 5, 10\}$ using shared noise, compared via per-sample L_2 distance and FID at each NFE level.

11 Implementation Details

11.1 Software Stack

Python 3.11.14, PyTorch 2.10+cu128, MONAI 1.5.2, PyTorch Lightning 2.6.1, OmegaConf/Hydra, HDF5 (h5py), Python logging with Rich handler + TensorBoard.

11.2 Gated Phase System

The project is implemented in 9 gated phases (0–8). Phase N+1 cannot begin until Phase N's critical tests all pass (Table 20).

Table 22: Phase status.

Phase	Title	Status	Tests
0	VAE Validation	Complete	7/7
1	Latent Pre-computation	Complete	7/7
2	Toy Experiment (Torus)	Complete	6/6 + 2 info
3	MeanFlow Loss + 3D UNet	Complete	28/28
4	Training on Brain MRI	Complete	12+
5	Generation + Evaluation	Code complete	11/11 local
6	Ablation Runs	Planned	—
7	LoRA Fine-Tuning (FCD)	Planned	—
8	Paper Figures	Planned	—

11.3 Compute Infrastructure

- **Local:** RTX 4060 8 GB — development, testing, analysis.
- **Picasso:** 4 nodes × 8× A100-40GB — training, encoding, evaluation. SLURM with `-constraint=dgx`.

12 Novel Contributions

Contribution 1: First MeanFlow for 3D Medical Image Synthesis. We achieve 1-NFE generation of 192^3 brain MRI volumes in the latent space of a frozen MAISI VAE—a **50–1000× reduction in sampling cost** compared to existing methods (DDPM: 1000; flow matching: 10–50; rectified flow: 5–50 steps). Best 2.5D FID: **11.67**, trained in ∼51 hours on 6× A100 GPUs.

Contribution 2: Per-Channel L_p Loss in Latent MeanFlow. We extend the SLIM-Diff per-channel L_p loss [6] from pixel-space DDPM to latent-space MeanFlow, providing a framework for ablation over $p \in \{1.0, \ldots, 3.0\}$.

Contribution 3: x-Prediction vs u-Prediction in Latent 3D UNets. First investigation of this dichotomy (established by pMF for 2D ViTs) in latent space with 3D UNets. We document

the critical incompatibility: **x-prediction + FD-JVP is numerically unstable** due to the $1/t$ singularity.

Contribution 4: iMF Dual-Head in Medical Context. First application of the iMF dual-head architecture outside natural images. The v-head adds ∼228 K parameters (<0.13%) and is disabled at inference.

Contribution 5 (Planned): LoRA Fine-Tuning for Joint Image-Mask Synthesis. Phase 7 will demonstrate LoRA fine-tuning for joint FLAIR MRI + FCD mask synthesis in a data-scarce setting (∼50–100 cases).

13 Current Status and Next Steps

13.1 Current State (February 2026)

- **Phases 0–4:** Complete. Best model trained (x-pred + exact JVP, best FID 11.67 at epoch 388, early-stopped at epoch 690).
- **Phase 5:** Code complete, 11/11 local tests pass. Awaiting Picasso execution for latent generation at NFE$\in \{1, 2, 5, 10\}$, VAE decoding, and full quantitative evaluation.
- **Phases 6–8:** Planned.

13.2 Key Findings

1. **Early stopping was effective:** Best FID at epoch 388, used only 46% of planned 1,500 epochs. Total training: 51.4 hours.
2. **1-NFE quality gap is significant:** NFE=1 is blurry; NFE≥2 is substantially sharper.
3. **Variance under-estimation:** Generated std ∼0.78 vs true ∼0.97–1.02. The 20–25% gap is the primary quality bottleneck.
4. **v-head converges fast:** $\cos(\hat{v}, v_c)$ reaches 0.39 by epoch 50 (85% of final 0.44).
5. **SWD anti-correlates with FID** beyond early training.
6. **Gradient clipping fraction:** 90% → 5%, confirming stable dynamics by epoch ∼300.

13.3 v2 Training Improvements

Based on the v1 training diagnostics (28,980 optimiser steps, 690 epochs), we identified three failure modes and designed targeted improvements. All changes are hyperparameter tweaks or minor code modifications; no architectural changes are introduced.

1. Boundary-aware time sampling (Section 6.2). The logit-normal distribution assigns $< 0.04\%$ of training samples to the 1-NFE region ($t \approx 1, r \approx 0$). We inject $\alpha = 10\%$ of each batch from $\mathcal{U}([0.95, 1] \times [0.001, 0.05])$ to provide direct training signal at the 1-NFE operating point.

2. Extended training with improved checkpointing. Max epochs increased from 1,500 to 3,000 (∼126K steps, 4.3× v1 budget). Early-stopping patience raised from 10 to 30 FID evaluations, with FID computed every validation epoch (every 10 training epochs) instead of every 3rd. The best-FID checkpoint callback retains the top-3 models (previously top-1) for rollback and post-hoc analysis.

3. Learning rate warmup. Linear warmup over 1,000 steps (∼24 epochs), addressing the 90.5% gradient clipping fraction observed at epoch 0. Justified by the large effective batch size ($B_{\text{eff}} = 132$) per Goyal *et al.* [16].

4. Inference-time norm correction. Post-training calibration of a scalar γ (Section 7.1) to compensate for compound velocity norm overshoot ($\|V_\theta\|/\|v_c\| = 1.16$ at epoch 689).

5. Per-time-bin cosine diagnostics. Cosine similarity $\cos(V, v_c)$ and $\cos(\tilde{v}, v_c)$ logged per time bin (near-data, mid, near-noise) for finer-grained convergence monitoring.

13.4 Open Questions

- Does the L_p exponent effect from pixel space transfer to latent space?
- How much does VAE smoothing degrade MeanFlow volumes vs multi-step methods?
- Can LoRA fine-tuning on ∼50–100 FCD cases produce anatomically plausible lesion synthesis?
- Can the variance under-estimation (∼75% of true std) be mitigated by post-hoc rescaling or loss modifications?

[1] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, "Mean Flows for One-step Generative Modeling," *NeurIPS*, 2025 (Oral). arXiv:2505.13447.

[2] Z. Geng, Y. Lu, Z. Wu, E. Shechtman, J. Z. Kolter, and K. He, "Improved Mean Flows: On the Challenges of Fastforward Generative Models," arXiv:2512.02012, 2025.

[3] Y. Lu, S. Lu, Q. Sun, H. Zhao *et al.*, "One-step Latent-free Image Generation with Pixel Mean Flows," arXiv:2601.22158, 2026.

[4] P. Guo, C. Zhao, D. Yang *et al.*, "MAISI: Medical AI for Synthetic Imaging," *WACV*, 2025. arXiv:2409.11169.

[5] C. Zhao, P. Guo, D. Yang *et al.*, "MAISI-v2: Accelerated 3D High-Resolution Medical Image Synthesis with Rectified Flow," arXiv:2508.05772, 2025.

[6] M. Pascual-González *et al.*, "SLIM-Diff: Shared Latent Image-Mask Diffusion with Lp Loss for Data-Scarce Epilepsy FLAIR MRI," arXiv:2602.03372, 2026.

[7] M. Yazdani *et al.*, "Flow Matching for Medical Image Synthesis," *MICCAI*, 2025. arXiv:2503.00266.

[8] Z. Dorjsembe *et al.*, "Semantic 3D Brain MRI Synthesis with Channel-Wise Conditioning," *IEEE JBHI*, 2024.

[9] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," *ICLR*, 2023.

[10] X. Liu, C. Gong, and Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," *ICLR*, 2023.

[11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," *CVPR*, 2022.

[12] E. Hu, Y. Shen, P. Wallis *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022.

[13] B. Billot *et al.*, "SynthSeg: Segmentation of Brain MRI Scans of Any Contrast and Resolution," *Medical Image Analysis*, 2023.

[14] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS*, 2020.

[15] R. Puglisi *et al.*, "Brain Latent Progression (BrLP)," *Medical Image Analysis*, 2025.

[16] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour," arXiv:1706.02677, 2017.

[17] S. Lin, B. Liu, J. Li, and X. Yang, "Common Diffusion Noise Schedules and Sample Steps are Flawed," *WACV*, 2024.